■ GIT CLASS 1 — INTERACTIVE NOTES ■

Git kya hai aur kaise kaam karta hai?

Created: 12 May 2026 | Student: Jyoti | Focus: Hands-on Learning

1■■ THEORY — Real-Life Example Se Samjho

Novel Likhi toh kya hota?

Socho tumne ek **novel likh rahi ho**:

✍■ **Draft 1** → **Draft 2** → **Draft 3**... aur phir tumhe yaad nahi kab kya change kiya
■ Kya problem hai? Purani version wapas nahi la sakte

■ **Git hota toh**: Har draft ka poora history save hota! Jab chaho purana version wapas la sakte.

Yahi Git karta hai — CODE ke liye!

■ Git = Version Control System (VCS)

Matlab: Tumhare code ke har change ka complete history save rahta hai. Kabhi bhi purana version wapas la sakte, kisi bhi change ko dekhna ho, ya multiple versions parallel mein develop kar sakte ho (branches ke through).

2■■ GIT KE 3 AREAS — Sabse Important Concept

Git mein code ka safar 3 stages se hota hai. Har stage ka ek alag purpose hai:

AREA	NAME	ANALOGY	KAISE KAAM KARTA HA
1■■	Working Directory	Tumhara room (saman bikhra hua)	Files edit karte ho
2■■	Staging Area	Bag pack karna	git add se select
3■■	Repository	Bag lock, trip pe	git commit - save

```
■ Git ka Flow:
■ Working Directory ↓ git add ↓ ■ Staging Area ↓ git commit ↓ ■■ Repository
```

3■■ PRACTICAL — Step by Step

■ STEP 1: VS Code Terminal Kholna

Ctrl + ` ya Terminal → New Terminal

```
git --version
```

■ **Expected: git version 2.43.0 (ya koi aur)**

■ STEP 2: Git Ko Naam Batao

```
git config --global user.name "Jyoti" git config --global user.email "rajeshware321@gmail.com"
```

```
git config --list (verify ke liye)
```

■ STEP 3: Project Folder Banana

```
cd Desktop mkdir git-practice cd git-practice code .
```

■ STEP 4: Git Init

```
git init
```

■■

```
ls -a (check karo)
```

■ STEP 5: Pehli File aur Commit

```
echo "# My First Git Project" > README.md
```

```
git status
```

```
git add README.md
```

```
git commit -m "feat: add README file"
```

■ **Pehla commit complete!**

■ STEP 6: History Dekho

```
git log --oneline
```

■ STEP 7: Changes Dekho

```
echo "print('Hello Jyoti!')" > app.py git add . git commit -m "feat: add app.py"
```

```
git diff HEAD~1
```

4■■ ■ PRACTICE CHECKLIST

#	TASK	COMMAND
1	Git version	git --version
2	Config setup	git config --global user.name "Jyoti"

3	Folder create	mkdir git-practice && cd git-practice
4	Git init	git init
5	File create	echo "text" > file.txt
6	Check status	git status
7	Stage files	git add .
8	Save changes	git commit -m "message"
9	View history	git log --oneline
10	See changes	git diff

5■■ ■ Common Doubts

Q1: git add . vs git add filename?
git add . = Sab files stage karo
git add file.js = Sirf ek file stage karo

Q2: Commit message kaise likhen?
Present tense: 'add feature' (NOT 'added feature')
feat: (feature), fix: (bug), docs: (documentation)

Q3: Galti se commit message diya?
git commit --amend -m "correct" (push se pehle!)

Q4: .git folder important kyun?
.git = complete history + configuration

6■■ ■ Key Concepts

Repository: Project folder + .git
Commit: Checkpoint/snapshot (unique ID)
Branch: Separate timeline (main/develop)
HEAD: Current commit position
Staging Area: Pre-commit selection
Working Directory: Actual folder
diff: Changes (Red=removed, Green=added)
log: Complete commit history

7■■ ■ Next Steps

■ **Today: Practice Karo**
• Naya folder → git init → files → commits → git log

■ **Commands Yaad Karo**
• git init, git add, git commit, git status, git log, git diff

■ **Class 2 Preview:**
• Branches: Parallel development
• git checkout: Switch branches
• git merge: Combine branches
• Team collaboration!

■ Remember: Git seekhna essential hai!
"Practice + Consistency = Git Master ■"